MIDNIGHT BUILDATHON · WAVE 3

Disclosure Ledger

Private proof of prior discovery for security researchers

Compact contract | 4 zero-knowledge circuits | Midnight dual public / private ledger | Apache-2.0

tidelit4 (AI-assisted build, disclosed)

github.com/tidelit4/disclosure-ledger

THE PROBLEM

"Duplicate."

Bug-bounty programs pay the first reporter. When two researchers find the same bug, the later report is closed as a duplicate, and the researcher has no way to show they found it first.

Proving it today means publishing something: a dated post, a Git commit, a hash on a public chain. That leaks the bug, is self-asserted, or ties every finding to one identity.

What a researcher needs

- a time bound **enforced by the network**, not claimed
- an entry that **reveals nothing** about the bug
- entries that **cannot be linked** to each other or to them
- a check that **one vendor** can run without telling anyone

HOW IT WORKS

Commit privately. Prove to one vendor. Disclose only when you choose.

1

Commit privately

`commitReport(notAfter)` publishes a commitment to `sha256(report)` with a random salt, a per-entry owner key, a global ordinal and a deadline the network enforces.

Report, digest, salt and secret key stay in private state.

2

Prove to one vendor, off-chain

The researcher sends the vendor the report and salt. The vendor hashes it with any tool, recomputes the commitment and reads the entry: "*on chain before T, entry #N*".

No transaction. Nobody else learns the check happened.

3

Disclose selectively, on chain

`proveMatch(commitment, vendorDigest)` proves in zero knowledge that the entry opens to the digest the vendor published. Then the owner can `markResolved`.

The salt and the report never touch the chain.

Entry lifecycle: committed → disclosed → resolved or committed → withdrawn

THE COMPACT CONTRACT

Four zero-knowledge circuits

Circuit	Who	Becomes public	Proved in zero knowledge
`commitReport(notAfter)`	anyone	commitment, owner key, ordinal, `notAfter`	block time < `notAfter`; commitment = `persistentCommit(digest, salt)` of the private draft; not already on the ledger
`proveMatch(c, vendorDigest)`	owner	status *disclosed*, `vendorDigest`	caller's secret key derives the entry's owner key; private opening matches the commitment; its digest equals `vendorDigest`
`markResolved(c)`	owner	status *resolved*	ownership; entry is *disclosed*
`withdraw(c)`	owner	status *withdrawn*	ownership; entry is *committed*

Pure helpers `reportCommitment` and `ownerKey` are exported to TypeScript, so a vendor hashes exactly as the circuit does. Witnesses: `localSecretKey()`, `draftReport()`, `reportOpening(c)`.

PRIVACY PROPERTIES

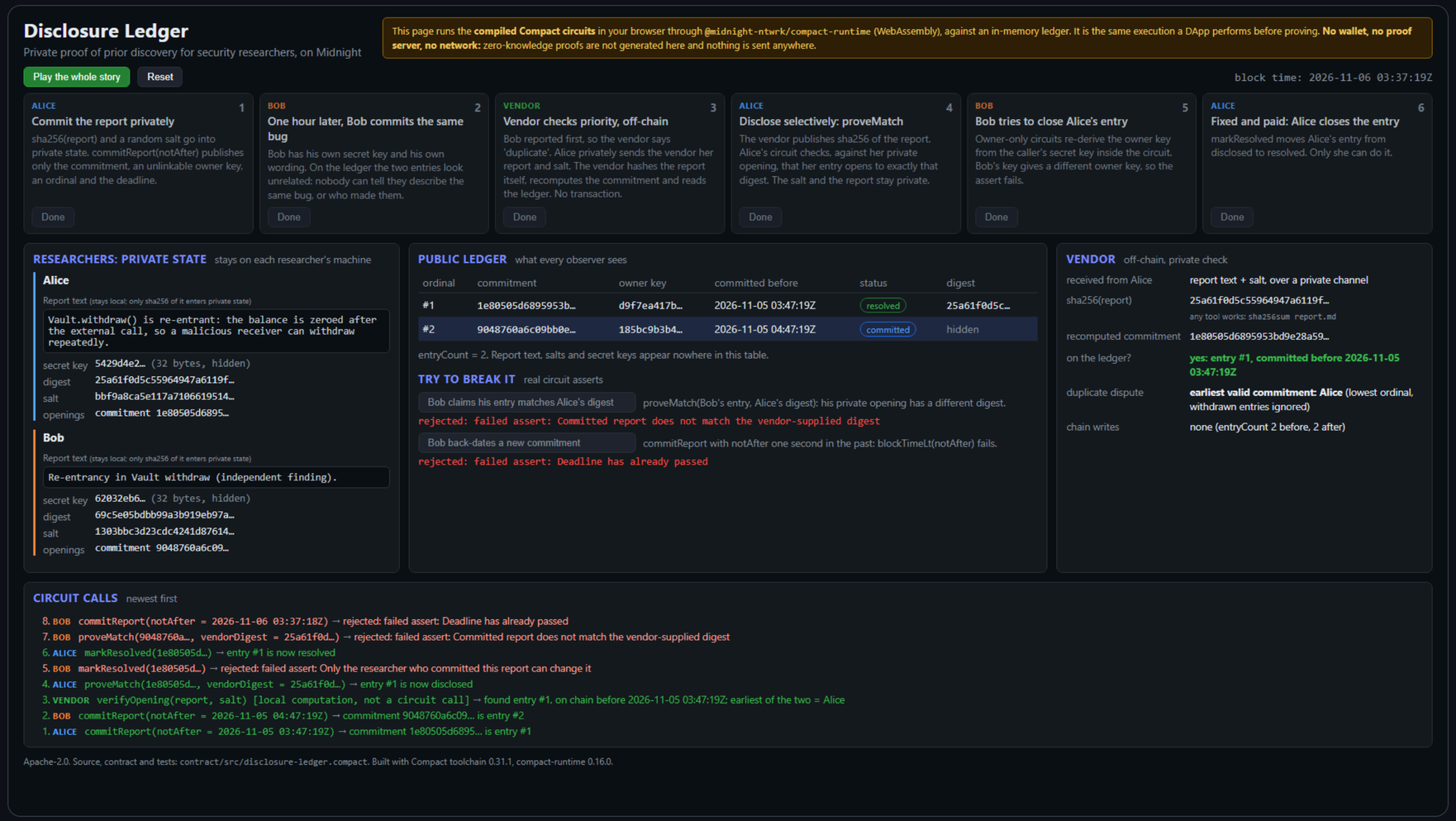
What is public, what stays private, and why

Data	Where
report text	researcher's disk only
salt (32 random bytes)	private state, or one vendor
secret key	private state
digest = sha256(report)	private until `proveMatch`
commitment	public: the anchor
owner key (per entry)	public, unlinkable
ordinal, `notAfter`	public: provable priority

- **Time is enforced, not claimed.** `blockTimeLt(notAfter)`: the ledger rejects a late commit.
- **Commitments, not hashes.** The salt blinds the digest, so a guessed report cannot be confirmed.
- **Unlinkable ownership.** Owner key = hash(domain, commitment, secret key), different for every entry.
- **Stolen openings are useless on chain.** Ownership is re-derived from the secret key inside the circuit.

DEMO

The compiled circuits, running in the browser



Static page in `ui/`. Every button calls the compiled contract through `compact-runtime` and its WebAssembly runtime.

Alice commits, Bob commits later, the vendor checks off-chain, Alice proves the match.

Three attacks are rejected by real circuit asserts. No wallet or proof server: proofs are not generated in the demo.

WHAT IS VERIFIED TODAY

Checked in CI, and what is not done yet

✓ Compiles with Compact toolchain **0.31.1** (language 0.23), the one Midnight's public networks run (ledger 8). Prover and verifier keys for all 4 circuits.

✓ **24 / 24** simulator tests: deadlines, duplicates, circuit/TypeScript hash agreement, unlinkable keys, stolen and tampered openings, lifecycle, dispute order.

✓ The same source compiles with toolchain **0.36** (language 0.28, ledger 9): the upgrade is a version bump.

✓ Browser smoke test: headless Chrome runs the full story and three attacks against the compiled contract.

! Not yet deployed to Preprod: funding needs the faucet's Cloudflare captcha, and Preprod RPC/indexer now need a Blockfrost account. Both are human steps, so CI cannot do it alone.

! No wallet or proof-server integration yet, so the demo runs the circuits without generating proofs.

! `notAfter` is an upper bound; inside one window the ordinal decides. A vendor must still judge whether two reports are the same bug.

ROADMAP

From working contract to a tool researchers use

1 · Live on Preprod

Deploy the contract; connect the web UI to the Lace wallet and a proof server through midnight-js, so every button sends a real proven transaction.

2 · Vendor-scoped disclosure

Prove the match to one vendor's public key without publishing the digest at all.

3 · Private reputation

Prove "I own N resolved entries" without revealing or linking which ones.

4 · Platform integration

A bounty platform checks the ledger before it marks a report as a duplicate.

TEAM

tidelit4

AI-assisted build, disclosed. The contract, tests, web UI, this deck and the demo video were written with an AI coding assistant (Claude, by Anthropic) working in tidelit4's repository. Every result shown is produced by code in the repository and checked in CI.

Code: `github.com/tidelit4/disclosure-ledger` · Apache-2.0

Run it: `npm ci --legacy-peer-deps && npm run compact && npm test && npm run ui:build && npm run ui:serve`

Deck and demo video: `docs/deck.pdf` · `docs/video/disclosure-ledger-demo.mp4`

Built on Midnight. Contract structure, owner-key pattern and simulator approach follow midnightntwrk/example-bboard and example-counter (Apache-2.0).